

AI Quiz Generator — Product Requirements & Technical Specification

1. Project Overview

Working name: AI Quiz Generator

Core idea: A user uploads a book/document (PDF, TXT, DOCX). The system reads the content, and — based on a selected learner profile (age/level) — generates an MCQ quiz. The user (or their child) attempts the quiz and sees the results.

Example use case: A parent/teacher uploads a children's book, selects a child profile (age 6, level: Beginner), the system generates 10 MCQs, the child answers, and a score/result is shown.

The system is not limited to children's books — any supported document can be used to generate a quiz. It also includes a dedicated, verified module for Quran and Islamic content (see Section 12+).

2. User Roles & Core Entities

User can:

- Sign up / Log in
- Create/manage learner profiles
- Upload books
- View book library
- Generate quizzes
- Attempt quizzes
- View results

Learner Profile (child/profile concept): One account can have multiple learner profiles.

```
Ali's Account
└─ Profiles
    └─ 👤 Ahmed — Age 6 — Level: Beginner
    └─ 👤 Sara — Age 10 — Level: Medium
```

Quiz generation flow references a specific learner profile:

Select Learner → Ahmed
Select Book → The Jungle Book
Difficulty → Beginner
Number of Questions → 10
Generate Quiz

3. Difficulty Levels

Keep it simple — do not create 5-6 confusing levels.

Final levels used in UI:

- Beginner
- Basic
- Medium
- (Advanced — optional, add later if needed)

4. Book Upload (General Books)

Supported formats (MVP):

- PDF (highest priority)
- TXT
- DOCX

Upload flow:

Upload Book → Select File → Uploading... → Processing... → Book Ready

Book Library example:

My Books

- The Jungle Book (PDF, 120 pages, uploaded Sep 23)
- Harry Potter (PDF, 340 pages, uploaded Sep 20)

Important: Quiz generation does NOT happen immediately on upload. Flow must be: Upload → Extract → Process → Store → Ready, then the user separately configures and generates a quiz.

5. Quiz Generation Flow (General Books)

User Login

- Select Learner Profile
- Select Book
- Select Difficulty
- Select Number of Questions
- (Optional) Select Chapter/Page Range
- Generate Quiz
- AI analyzes relevant book content
- AI generates MCQs
- Validate questions
- Quiz Ready
- User Attempts Quiz
- Submit
- Score + Results

6. Quiz Configuration Screen

Generate Quiz

Learner: [Ahmed]

Book: [The Jungle Book]

Difficulty:

- ☐ Beginner
- ☐ Basic
- ☒ Medium

Number of Questions: [10]

Content:

- ☐ Entire Book
- ☐ Specific Pages
- ☐ Specific Chapter

[Generate Quiz]

Question type (MVP): MCQ only. Later additions: True/False, Fill in the blanks, Short answer.

7. MCQ Structure

Question 1 of 10

What did the boy find in the forest?

- A. A book
- B. A tiger
- C. A house
- D. A river

[Previous] [Next] (last question shows [Submit Quiz])

8. Result Screen

Quiz Completed 🎉

Ahmed

Score: 8 / 10 (80%)

Correct Answers: 8

Wrong Answers: 2

Difficulty: Beginner

Book: The Jungle Book

[Review Answers] [Back to Dashboard]

Review screen (per question):

Question 1: What did the boy find?

Your answer: B. A tiger

Correct answer: B. A tiger

✓ Correct

Question 4:

Your answer: A. River

Correct answer: C. House

✗ Incorrect

9. Dashboard

Welcome, Ali

Your Profiles: Ahmed (6), Sara (10)

Your Books: 3 Books

Recent Quizzes:

- The Jungle Book – Ahmed – 8/10
- Physics Fundamentals – Ali – 7/10

[+ Add Profile] [+ Upload Book] [+ Generate Quiz]

10. Profile System

Account profile: Name, Email, Password

Learner profile: Name, Age, Learning Level, Avatar

The learner profile influences AI generation:

- 6-year-old + Beginner → simpler vocabulary and concepts
- 18-year-old + Medium → more complex questions

11. AI Generation Logic (Core)

The AI must NOT be told simply "generate 10 questions about this book." It must receive full context:

Book Content: [chunked relevant content]

Learner:

Age: 6

Level: Beginner

Difficulty: Basic

Question Count: 10

Generate: MCQs

Requirements:

- Questions must be based only on provided book content.
- Do not invent facts.
- Each question must have 4 options.
- Only one option must be correct.
- Wrong options must be plausible.
- Language must match learner level.
- Return structured JSON.

Expected JSON response format:

json

```
{
  "questions": [
    {
      "question": "Where did Ali go?",
      "options": ["School", "Forest", "Market", "Park"],
      "correctAnswer": 1,
      "explanation": "The book states that Ali went to the forest."
    }
  ]
}
```

Important: The correct answer must never be exposed to the frontend during quiz attempt — only after submission.

12. AI Hallucination Problem — Content Pipeline

The AI must never generate questions about content that isn't in the book. Architecture:

PDF

- Text Extraction
- Clean Text
- Chunking
- Relevant Content Retrieval
- AI
- MCQs
- Validation
- Database

For large books, do not send the entire book in a single AI request — divide content into chunks:

Book

- └─ Chapter 1 → Chunk 1, Chunk 2, Chunk 3
- └─ Chapter 2 → Chunk 1, Chunk 2

This can later be extended into RAG / vector search.

13. Recommended Architecture

Layer	Choice
Frontend	React + Tailwind CSS
Backend	Node.js + Express.js
Database	MongoDB
Auth	JWT + bcrypt
File Storage	Cloudinary / S3-compatible (or local for dev)
AI Provider	Configurable via env vars: <code>AI_API_KEY</code> , <code>AI_MODEL</code>

14. Database Structure (General Content)

User

`_id, name, email, passwordHash, createdAt, updatedAt`

LearnerProfile

```
_id, userId, name, age, level, avatar, createdAt, updatedAt
```

Book

```
_id, userId, title, fileUrl, fileType, fileSize, pageCount,  
extractedText, processingStatus, createdAt, updatedAt
```

```
processingStatus: UPLOADING | PROCESSING | READY | FAILED
```

Quiz

```
_id, userId, learnerProfileId, bookId, difficulty,  
questionCount, status, score, totalQuestions, createdAt, completedAt
```

Question

```
_id, quizId, questionText, options[], correctAnswer, explanation
```

QuizAnswer (for attempt history)

```
_id, quizId, questionId, selectedAnswer, isCorrect
```

15. API Structure (General Content)

Authentication

```
POST /api/auth/register  
POST /api/auth/login  
GET /api/auth/me
```

Profiles

```
POST    /api/profiles
GET     /api/profiles
GET     /api/profiles/:id
PUT     /api/profiles/:id
DELETE  /api/profiles/:id
```

Books

```
POST    /api/books/upload
GET     /api/books
GET     /api/books/:id
DELETE  /api/books/:id
```

Quiz

```
POST /api/quizzes/generate
GET  /api/quizzes
GET  /api/quizzes/:id
POST /api/quizzes/:id/submit
GET  /api/quizzes/:id/result
```

16. UI Pages List

1. Landing Page
 2. Login
 3. Register
 4. Dashboard
 5. Learner Profiles
 6. Create/Edit Learner
 7. Book Library
 8. Upload Book
 9. Book Details
 10. Generate Quiz
 11. Quiz Interface
 12. Quiz Result
 13. Quiz History
 14. Settings
-

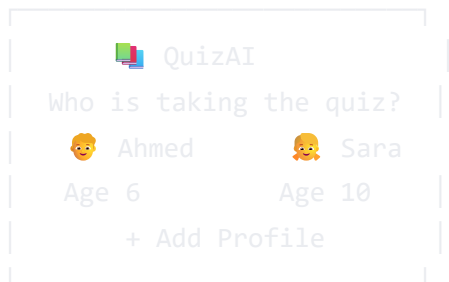
17. Design Direction

The client specifically asked for an **app-like** feel — not a traditional admin dashboard.

Design principles:

- Clean, modern, responsive
- Card-based UI
- Mobile-friendly
- Simple navigation
- Large buttons
- Readable typography
- Minimal clutter
- Especially child-friendly interface for learner profile selection

Example (profile selector):



18. Edge Cases (General Content)

Case	Handling
Empty/invalid PDF	"This file could not be processed."
Scanned PDF (no extractable text)	"This appears to be a scanned document. OCR processing is required."
Very large book	Show processing queue/status
AI generation failure	"Quiz generation failed. Please try again."
No relevant content found	"We couldn't generate enough questions from the selected content. Try selecting a larger section."
Duplicate questions	Reject via validation
Multiple correct answers in one question	Reject via validation
No correct answer marked	Reject via validation

19. Security Requirements

- Passwords hashed with bcrypt
- JWT authentication
- Protected API routes
- User can only access their own books/profiles/quizzes
- File type validation
- File size limit
- Environment variables for API keys
- Never expose AI API key to frontend
- Input validation on all endpoints
- Proper error handling
- Rate limiting on quiz generation endpoint
- Never expose correct answers before quiz submission

20. Content Categories Overview

The system supports two distinct content categories, each with different trust/verification requirements:

Book Type

- └ General Book (user-uploaded)
 - └ PDF/TXT/DOCX, any topic
 - └ Normal AI extraction + MCQ flow
- └ Islamic Content (admin-curated / verified source only)
 - └ Quran
 - └ Islamic Books (Hadith, Seerah, Tafseer, Fiqh, etc.)

User-facing selection at quiz creation:

What do you want to create a quiz from?

-  General Book (upload your own)
-  Islamic Library (Quran & verified Islamic books)

21. Quran Module (Verified Content Only)

Why Quran cannot follow the normal user-upload flow:

- Arabic text is sensitive to AI misreading/misparing — errors are unacceptable.
- Translation authenticity matters — the system cannot decide on its own which translation/scholar is authoritative.
- Verse order and exact wording must be accurate; AI hallucination is not acceptable here.

Approach: Quran is admin-seeded from a verified source — never user-uploaded.

Verified Quran API/Dataset (e.g. alquran.cloud, quran.com — admin-seeded once)

- Already structured (Surah, Ayah, Arabic text, Translation)
- When generating MCQs, AI receives ONLY verified translation text (never asked to parse/interpret raw Arabic)
- Every question stored with its verse reference (Surah:Ayah)

Quiz configuration screen (Quran):

Generate Quiz — Quran

Learner: [Ahmed]

Select:

- Specific Surah
- Juz
- Ayah Range

Translation: [admin-defined list of authentic translations]

Difficulty: Beginner / Basic / Medium

Question Count: [10]

[Generate Quiz]

Question types — factual/informational only, never interpretation-based:

- "This ayah belongs to which Surah?"
- "What is mentioned in this ayah?"
- "Was this Surah revealed in Makkah or Madinah?"
- Meaning-based MCQs sourced strictly from the verified translation (not the AI's own tafseer)

Database — QuranVerse collection:

```
_id
surahNumber
surahName
ayahNumber
arabicText
translationSource      (e.g. "Saheeh International")
translationText
juz
revelationPlace        (Makkah / Madinah)
```

Validation rule: AI-generated Quran questions should ideally go through human (admin/moderator) review before publishing. At minimum, show this disclaimer in the MVP:

⚠ Quiz questions are auto-generated from verified translations.
Please report any inaccuracy.

22. Islamic Books Module (Hadith, Seerah, Tafseer, Fiqh)

Less sensitive than Quran (no raw Arabic-translation risk at the same level), but still **not user-uploaded** — admin-curated only, to preserve source authenticity.

Admin uploads verified Islamic books (Bukhari, Muslim, Seerah books, etc.)
→ Normal PDF extraction + chunking (same pipeline as General Books)
→ AI quiz generation (same pipeline as General Books)
→ Difference: source library is "curated," not user-uploaded

Database — IslamicBook collection:

```
_id  
title  
author  
category      (Hadith / Seerah / Tafseer / Fiqh)  
fileUrl  
extractedText  
verifiedBy    (admin flag)  
createdAt
```

(Other fields — processingStatus, pageCount, etc. — same as general Book.)

23. Updated Book/Content Selection Flow

```
Generate Quiz  
  → Select Category  
      ├── My Uploaded Books → Book collection  
      └── Islamic Library  
          ├── Quran → QuranVerse collection  
          └── Islamic Books → IslamicBook collection
```

24. Updated API Endpoints (Islamic Content)

Quran

```
GET /api/quran/surahs
GET /api/quran/surah/:number
POST /api/quizzes/generate-quran (Surah/Ayah range based)
```

Islamic Books (admin-managed)

```
GET /api/islamic-books
GET /api/islamic-books/:id
POST /api/quizzes/generate-islamic-book
```

Admin only

```
POST /api/admin/islamic-books/upload
POST /api/admin/quran/seed
```

25. Development Phases

Phase 1 — Foundation

- React frontend
- Node/Express backend
- MongoDB
- Authentication
- Basic UI/layout

Phase 2 — Profiles

- Learner profiles (age, level, profile selection)

Phase 3 — Books

- Upload, library, PDF processing, text extraction

Phase 4 — AI Quiz

- Quiz configuration
- AI integration
- MCQ generation
- JSON validation
- General book quiz generation
- Quran quiz generation (Surah/Ayah based, verified translation only)

- Islamic books quiz generation (admin-curated library)
- Content-source-aware prompt engineering (different prompt rules for Quran vs. general text)

Phase 5 — Quiz

- Question navigation, answer selection, submit, scoring

Phase 6 — History

- Previous quizzes, results, profile performance

Phase 7 — Polish

- Responsive design, loading states, error handling, security, empty states, animations, final testing

26. MVP Demo Flow (Most Important — What to Show)

This is the complete flow that must work end-to-end for the MVP demo:

```
Register
  → Create child profile
  → Upload a PDF book
  → Book processed
  → Select child
  → Select difficulty
  → Select 10 questions
  → Generate Quiz
  → AI creates MCQs
  → Attempt quiz
  → Submit
  → Score
  → Review answers
```

If this flow works reliably end-to-end, the concept is fully demonstrated.

Notes for Antigravity

- Build in phases (see Section 25) rather than requesting the entire app in one prompt.
- Treat Quran and Islamic Books as a **separate, verified-only content pipeline** distinct from user-uploaded General Books — do not merge their upload/validation logic.
- Never let the AI parse or interpret raw Quranic Arabic; only feed it verified translation text for Quran-related question generation.

- Enforce all edge cases and validation rules listed in Sections 18 and 21 at the backend level, not just the UI.